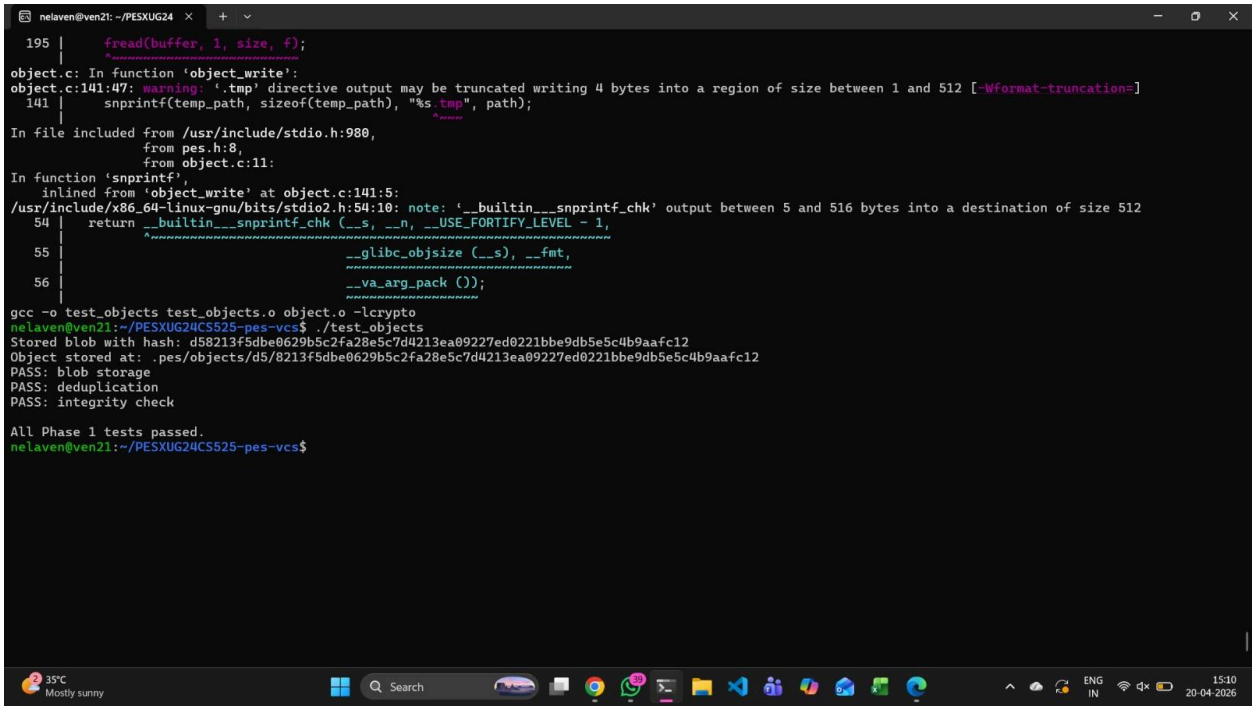
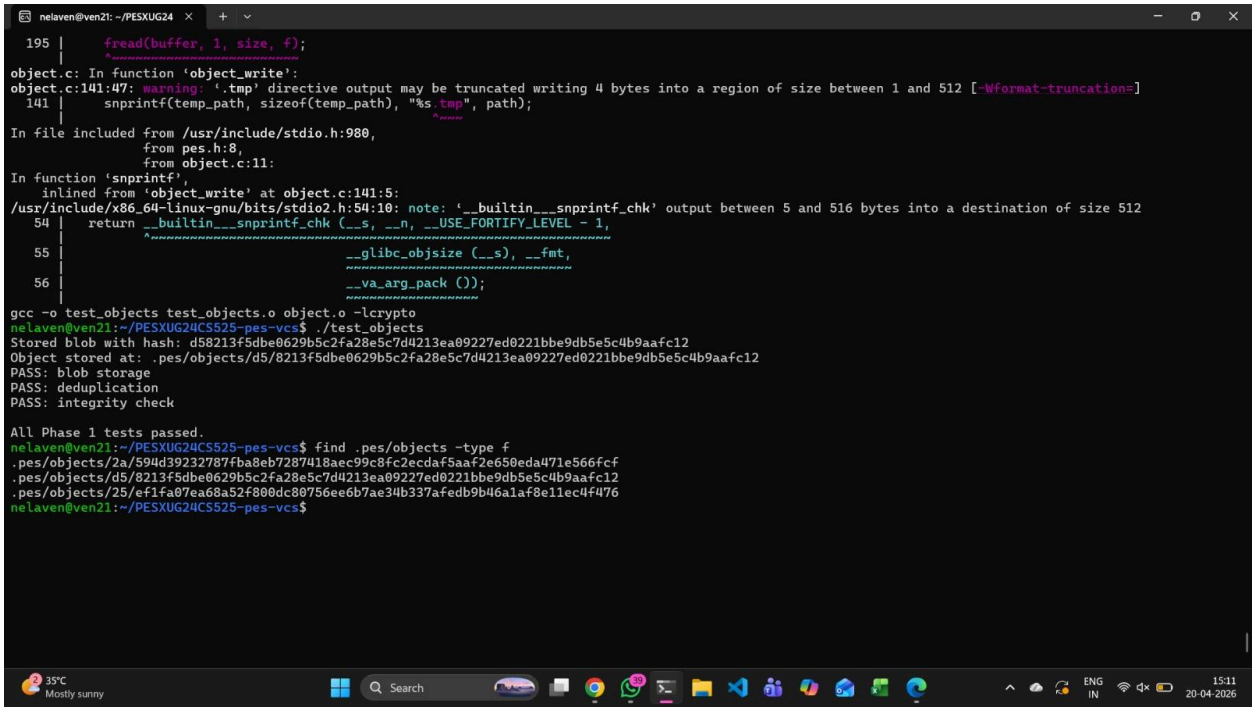
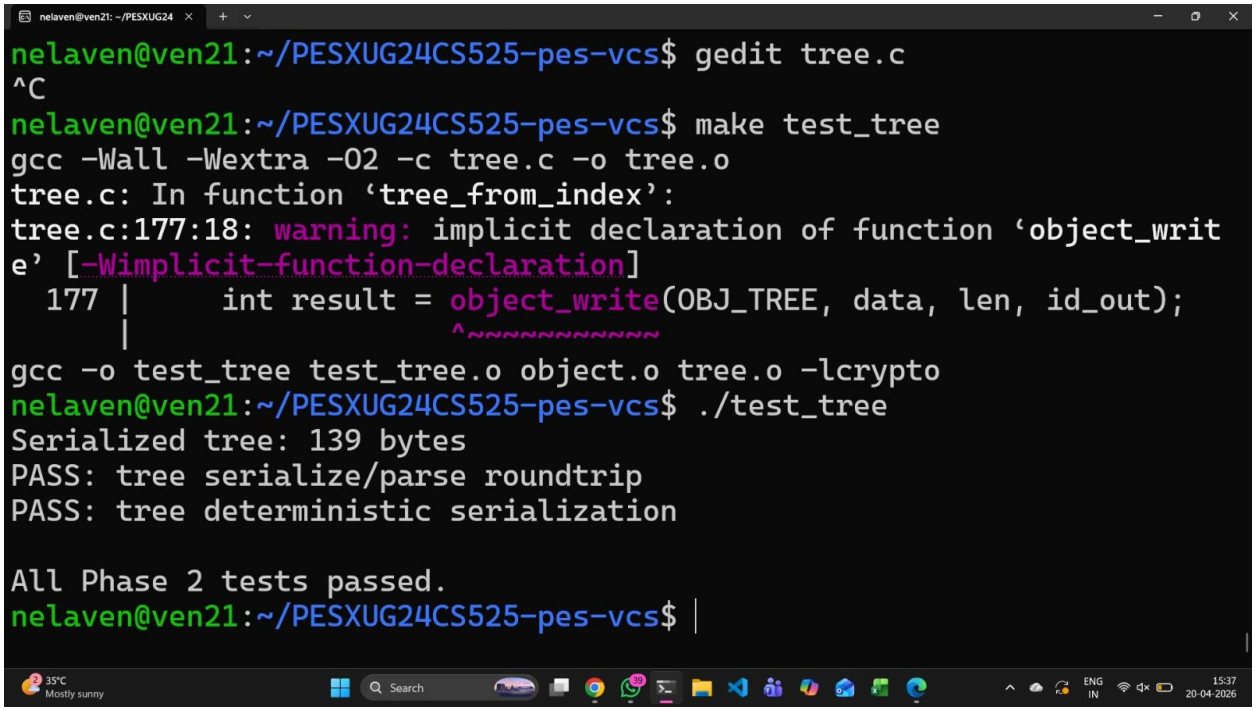
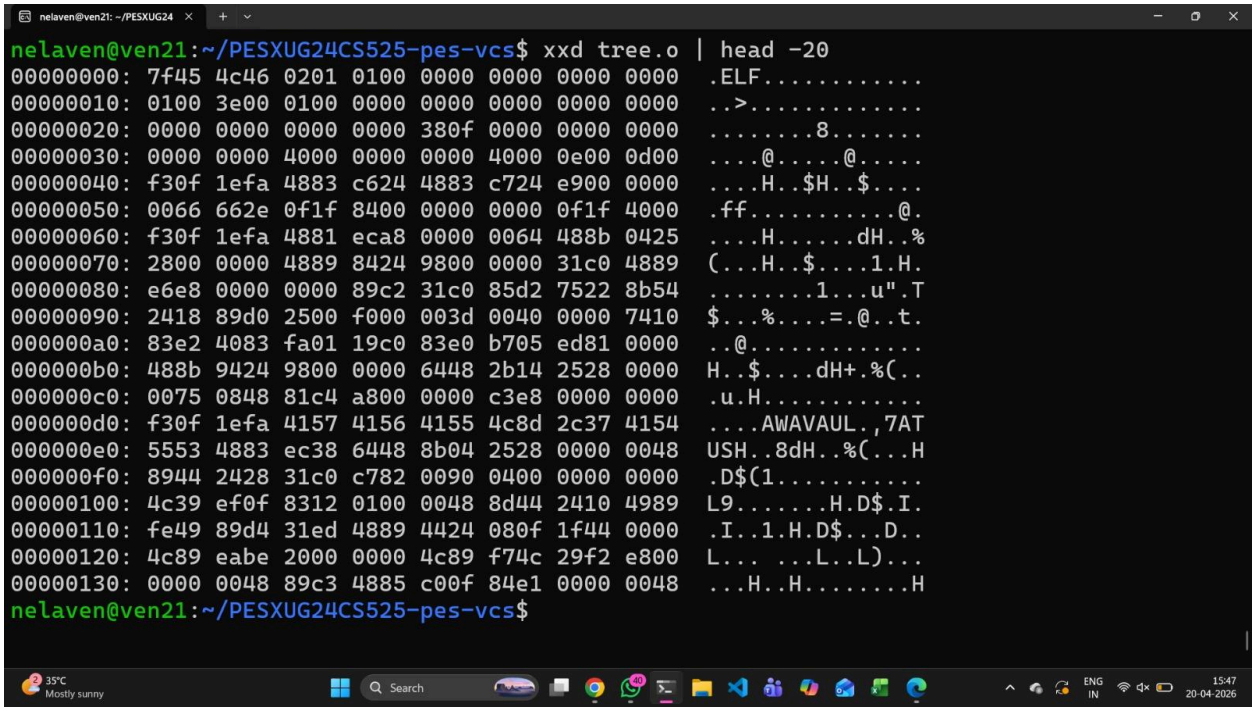

nelaven@ven21: ~/PESXUG24
195 | fread(buffer, 1, size, f);
object.c: In function 'object_write':
object.c:141:47: warning: '.tmp' directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
141 | snprintf(temp_path, sizeof(temp_path), "%s.tmp", path);
In file included from /usr/include/stdio.h:980,
from pes.h:8,
from object.c:11:
In function 'snprintf',
inlined from 'object_write' at object.c:141:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 5 and 516 bytes into a destination of size 512
54 | return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
55 | __glibc_objsize (__s), __fmt,
56 | __va_arg_pack ());
gcc -o test_objects test_objects.o object.o -lcrypto
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
nelaven@ven21:~/PESXUG24CS525-pes-vcs$
35°C
Mostly sunny
Search
ENG
IN
15:10
20-04-2026

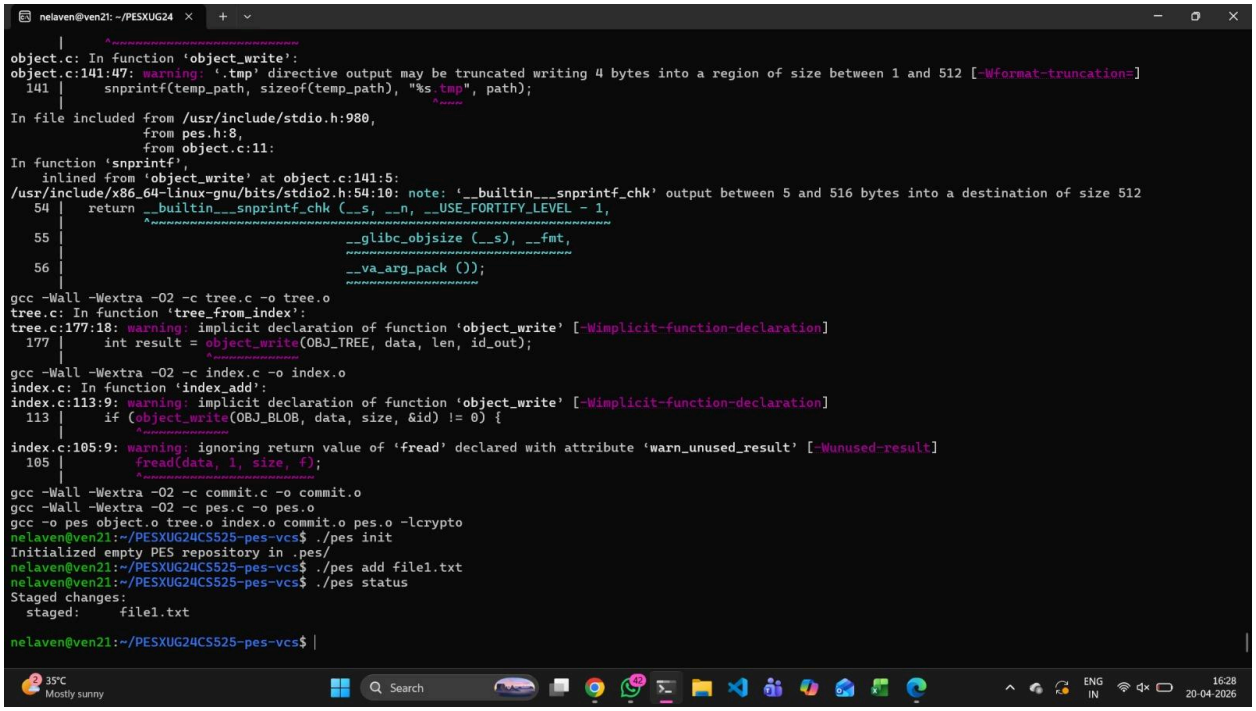

nelaven@ven21: ~/PESXUG24
195 | fread(buffer, 1, size, f);
object.c: In function 'object_write':
object.c:141:47: warning: '.tmp' directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
141 | snprintf(temp_path, sizeof(temp_path), "%s.tmp", path);
In file included from /usr/include/stdio.h:980,
from pes.h:8,
from object.c:11:
In function 'snprintf',
inlined from 'object_write' at object.c:141:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 5 and 516 bytes into a destination of size 512
54 | return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
55 | __glibc_objsize (__s), __fmt,
56 | __va_arg_pack ());
gcc -o test_objects test_objects.o object.o -lcrypto
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
nelaven@ven21:~/PESXUG24CS525-pes-vcs$
35°C
Mostly sunny
Search
ENG
IN
15:11
20-04-2026

PHASE 1


nelaven@ven21:~/PESXUG24CS525-pes-vcs$ gedit tree.c
^C
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'tree_from_index':
tree.c:177:18: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
177 | int result = object_write(OBJ_TREE, data, len, id_out);
| ^~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
All Phase 2 tests passed.
nelaven@ven21:~/PESXUG24CS525-pes-vcs$


nelaven@ven21:~/PESXUG24CS525-pes-vcs$ xxd tree.o | head -20
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000 .ELF............
00000010: 0100 3e00 0100 0000 0000 0000 0000 0000 ..>.............
00000020: 0000 0000 0000 0000 380f 0000 0000 0000 ....8.......
00000030: 0000 0000 4000 0000 0000 4000 0e00 0d00 ....@.....@.....
00000040: f30f 1efa 4883 c624 4883 c724 e900 0000 ....H..$H..$....
00000050: 0066 662e 0f1f 8400 0000 0000 0f1f 4000 .ff...........@.
00000060: f30f 1efa 4881 eca8 0000 0064 488b 0425 ....H......dH..%
00000070: 2800 0000 4889 8424 9800 0000 31c0 4889 (...H..$....1.H.
00000080: e6e8 0000 0000 89c2 31c0 85d2 7522 8b54 ....1...u".T
00000090: 2418 89d0 2500 f000 003d 0040 0000 7410 $...%....=.@..t.
000000a0: 83e2 4083 fa01 19c0 83e0 b705 ed81 0000 ..@.............
000000b0: 488b 9424 9800 0000 6448 2b14 2528 0000 H..$....dH+.%(..
000000c0: 0075 0848 81c4 a800 0000 c3e8 0000 0000 .u.H............
000000d0: f30f 1efa 4157 4156 4155 4c8d 2c37 4154 ....AWAVAUL.,7AT
000000e0: 5553 4883 ec38 6448 8b04 2528 0000 0048 USH..8dH..%(...H
000000f0: 8944 2428 31c0 c782 0090 0400 0000 0000 .D$(1...........
00000100: 4c39 ef0f 8312 0100 0048 8d44 2410 4989 L9.......H.D$.I.
00000110: fe49 89d4 31ed 4889 4424 080f 1f44 0000 .I..1.H.D$...D..
00000120: 4c89 eabe 2000 0000 4c89 f74c 29f2 e800 L... ...L..L)...
00000130: 0000 0048 89c3 4885 c00f 84e1 0000 0048 ...H..H........H
nelaven@ven21:~/PESXUG24CS525-pes-vcs$

PHASE 2




```
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776682129 6 file1.txt
nelaven@ven21:~/PESXUG24CS525-pes-vcs$
```

PHASE 3

```
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ echo "hello" > file1.txt
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./pes add file1.txt
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./pes commit -m "first commit"
Committed: 6c4ccb8080be... first commit
```

```
nelaven@ven21:~/PESXUG24CS525-pes-vcs$ ./pes log
commit 6c4ccb8080be2b7b6e3b4dc8c8bac2dd488b2fb3c3ded0e6d60c3f89697918fb
Author: PES User <pes@localhost>
Date:   1776683355

    first commit

nelaven@ven21:~/PESXUG24CS525-pes-vcs$
```

PHASE 4

PHASE 5 AND PHASE 6 [QnA]

Phase 5: Branching and Checkout

Q5.1 — How does `pes checkout <branch>` work?

When switching branches, the following changes occur:

- The `.pes/HEAD` file is updated to point to the new branch
- The working directory is updated to match the target commit
- The index is reloaded to reflect the new tree

Steps involved:

1. Read the current HEAD to determine the current branch and commit
2. Identify the target branch or commit
3. Load the target commit's tree object
4. Update files in the working directory
5. Update `.pes/HEAD` to the new branch
6. Reload the index to match the new tree

Complexities:

- Handling modified (dirty) files
- Recursive directory updates
- Maintaining consistency if interrupted

Q5.2 — How are dirty file conflicts detected?

A file is considered conflicting if:

1. It exists in the index
2. Its current content differs from the stored hash
3. The target branch contains a different version

Detection process:

- Compute the hash of the file in the working directory
- Compare it with the index hash
- Compare it with the target branch hash

If all differ, a conflict is detected and checkout is prevented.

Q5.3 — What is Detached HEAD and how can it be recovered?

Detached HEAD occurs when HEAD points directly to a commit instead of a branch.

In this state:

- New commits are not associated with any branch
- They can become unreachable if not saved

Recovery methods:

Before leaving detached state:
pes branch recover-work

After leaving:

- Search `.pes/objects` for commit objects
- Identify dangling commits
- Create a new branch pointing to the required commit

Phase 6: Garbage Collection and Space Reclamation

Q6.1 — How to find and delete unreachable objects?

Garbage collection uses the **Mark and Sweep algorithm**.

Mark Phase:

- Start from all branch references
- Traverse commits, trees, and blobs
- Store all reachable objects

Sweep Phase:

- Traverse all objects in `.pes/objects/`
- Delete objects not present in the reachable set

Data Structure:

- A hash set is used for fast lookup of object IDs

Estimated scale:

- For large repositories, around 400,000 to 700,000 objects may be processed

---

Q6.2 — What is a race condition between GC and commit?

A race condition occurs when garbage collection runs while a commit is being created:

1. Commit writes a new object
2. GC runs before the commit completes
3. GC deletes the new object (thinking it is unused)
4. Commit references a deleted object

This leads to repository corruption.

How Git avoids this:

- Recently created objects are not deleted
- Lock files are used during operations
- Reachability is rechecked before deletion

Simplified solution:

- Avoid running GC during active writes
- Detect temporary or lock files before deletion

Conclusion

Phases 5 and 6 focus on advanced version control concepts such as branch management, conflict detection, and storage optimization. These mechanisms ensure data integrity, efficient storage, and reliable version tracking in a version control system.